GUIA TÉCNICO EP / DISCIPLINA 04

Transações e PostgreSQL

ACID, rollback, isolamento e validação em serviço real

MAPA DE ESTUDO

01 Propriedades ACID e invariantes

02 Rollback e teste local com SQLite

03 Conexão opcional ao PostgreSQL

04 Isolamento, MVCC e concorrência

05 Diagnósticos e limites da demonstração

Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

05 | Transações e PostgreSQL

GUIA TÉCNICO APROFUNDADO • ANÁLISE, CÓDIGO E INTERPRETAÇÃO

Objetivo e escopo

Estudo independente baseado no notebook aprovado. Os exemplos locais usam SQLite e dados sintéticos; PostgreSQL, MongoDB e Cassandra requerem serviços de laboratório para verificação real.

Conceito central: Atomicidade e isolamento

Uma transação deve manter invariantes mesmo se uma instrução falhar. COMMIT confirma o conjunto; ROLLBACK desfaz alterações não confirmadas. Considere anomalias de concorrência e níveis de isolamento na implementação real do PostgreSQL.

Aplicação, ressalvas e limites

A demonstração com SQLite comprova rollback local. Ela não valida MVCC, bloqueios ou isolamento de uma instância PostgreSQL; isso depende do laboratório externo opcional.

ACID na prática

Atomicidade: tudo ou nada. Consistência: preservação de invariantes do banco. Isolamento: transações concorrentes devem seguir nível especificado. Durabilidade: alterações confirmadas persistem após falhas; efeitos dependem de configurações do SGBD.

Exemplo do notebook — execução local

```
import sqlite3
conn = sqlite3.connect(':memory:')
conn.execute('PRAGMA foreign_keys=ON')
conn.executescript((Path('sql')/'01_schema_sqlite.sql').read_text(encoding='
    utf8'))
leituras = pd.read_csv(DATA/'leituras_sensores.csv')
leituras.to_sql('leituras',conn,if_exists='append',index=False)
print('SQLite inicializado:',conn.execute('SELECT COUNT(*) FROM leituras').f
    etchone()[0], 'leituras')
```

Rollback local no SQLite

Demonstração de cancelamento de inserção; SQLite não substitui os laboratórios de concorrência do PostgreSQL.

Exemplo do notebook — execução local

```
antes=conn.execute('SELECT COUNT(*) FROM leituras').fetchone()[0]
conn.execute('BEGIN') if not conn.in_transaction else conn.commit()
conn.execute("INSERT INTO leituras VALUES (999,1,'2025-02-01',22,50)")
conn.rollback()
depois=conn.execute('SELECT COUNT(*) FROM leituras').fetchone()[0]
print('Linhas antes/depois:',antes,depois);assert antes==depois
```

Prática opcional - PostgreSQL real

Ative o serviço com docker compose -f infra/compose.yaml --profile postgres up -d e configure POSTGRES_PASSWORD via .env na pasta infra. O código abaixo não declara sucesso sem conexão; faz uma checagem explícita e só executa quando RUN_LIVE_DB=1.

Exemplo do notebook — laboratório opcional, exige serviço isolado

```
# A execução local não ativa conexões automaticamente.
# Para laboratórios opcionais, configure RUN_LIVE_DB=1 antes de iniciar o ke
    rnel,
# depois de verificar que a instância é EXCLUSIVAMENTE de estudos.
import os
print('Laboratório externo:', 'habilitado' if os.getenv('RUN_LIVE_DB') == '1
    ' else 'desabilitado (padrão seguro)')
```

Exemplo do notebook — laboratório opcional, exige serviço isolado

```
import os
if os.getenv('RUN_LIVE_DB')=='1':
 import psycopg
 senha=os.getenv('POSTGRES_PASSWORD')
 assert senha,'Defina POSTGRES_PASSWORD no ambiente do Jupyter.'
 with psycopg.connect(host='localhost',port=5433,user=os.getenv('POSTGRES_US
    ER','estudos'),password=senha,dbname=os.getenv('POSTGRES_DB','biblioteca
    '),connect_timeout=5) as pg:
  print('Versão real:',pg.execute('SELECT version()').fetchone()[0])
else:
 print('Prática real não executada. Defina RUN_LIVE_DB=1 com o serviço ativo
    .')
```

Exercício resolvido - transferência

Faça uma transferência entre contas somente se o saldo de origem for suficiente; use transação e CHECK(saldo >= 0). O script sql/03_postgres.sql é uma base de estudo, não representa tratamento completo de concorrência.

Exemplo do notebook — execução local

```
print((Path('sql')/'03_postgres.sql').read_text()[:400])
```

--- ### Aprofundamento v2 - Atomicidade e idempotência

Uma transação garante atomicidade das operações no mesmo banco, mas não substitui modelagem de idempotência nem controle de concorrência. Em PostgreSQL, verifique o nível de isolamento e use tabelas exclusivamente de laboratório. O bloco abaixo funciona sem servidor e não altera arquivos persistentes.

Interpretação: compare o resultado com a teoria anterior; este exemplo local não substitui um laboratório de serviço externo.

Exemplo do notebook — execução local

```
import sqlite3
with sqlite3.connect(':memory:',isolation_level=None) as db:
    db.execute('CREATE TABLE contas(id INTEGER PRIMARY KEY, saldo INTEGER CH
    ECK(saldo>=0))')
    db.executemany('INSERT INTO contas VALUES (?,?)',[(1,100),(2,30)])
    db.execute('BEGIN')
    try:
        db.execute('UPDATE contas SET saldo=saldo-200 WHERE id=1')
        db.execute('UPDATE contas SET saldo=saldo+200 WHERE id=2')
        db.execute('COMMIT')
    except sqlite3.IntegrityError:
        db.execute('ROLLBACK')
```

```
assert db.execute('SELECT SUM(saldo) FROM contas').fetchone()[0] ==130
print('Rollback preservou os saldos:',db.execute('SELECT * FROM contas O
RDER BY id').fetchall())
```

Exercício de revisão

Exercício resolvido - revisão: identifique a principal hipótese do exemplo acima, explique por que ela importa e cite uma condição na qual a técnica falharia. Resposta orientadora: registre a pré-condição (tipo de dado, chave, restrição, volume ou topologia), teste-a explicitamente e explique os limites dos resultados observados.

Aprofundamento: decisões e estudo de caso

Transações e invariantes

Considere a transferência de 50 unidades entre duas contas. Se o débito ocorrer e o crédito falhar, o saldo total não pode mudar. Agrupar as instruções em transação possibilita desfazer todas as alterações não confirmadas. Restrições CHECK também podem impedir saldos negativos; a operação deve tratar a exceção, executar ROLLBACK e registrar adequadamente o evento.

O que o teste local prova

No SQLite, uma transação e seu rollback podem ser verificados por SELECT antes e depois. Esse teste comprova atomicidade observável naquele ambiente, não permite afirmar o comportamento de bloqueios, isolamento ou throughput do PostgreSQL. O pacote separa as experiências: SQLite como demonstração reproduzível, psycopg como acesso opcional a serviço real.

Isolamento em ambiente real

Níveis de isolamento controlam quais estados podem ser vistos por transações concorrentes e quais conflitos exigem reexecução. Em PostgreSQL, Repeatable Read e Serializable têm semânticas próprias. Para avaliá-los, seriam necessárias conexões simultâneas, transações explícitas e observação de resultados sob diferentes interleavings; uma única célula de notebook não substitui esse teste.

Exercício seguro

Implemente a transferência de dois saldos em SQLite com BEGIN, CHECK, tratamento de erro e ROLLBACK. Compare o saldo total antes e depois da falha proposital. Depois descreva um teste com duas sessões PostgreSQL, deixando explícito que sua execução depende de laboratório isolado e não ocorre automaticamente no Run All padrão.

Checklist de interpretação

- Defina a pergunta, unidade de análise e pré-condições da consulta.
- Registre chaves, cardinalidade, restrições e tratamento de valores ausentes.
- Execute os exemplos locais antes de habilitar serviços reais.
- Separe evidência observada de hipóteses e propriedades não testadas.

Leituras e arquivos de apoio

Consulte o notebook correspondente, os scripts em sql/, os datasets sintéticos e a documentação oficial dos SGBDs. Os resultados locais não substituem testes de integração com serviços externos.